

TUGAS BESAR ARSITEKTUR PERANGKAT LUNAK
ANALISIS GAYA ARSITEKTUR PADA APLIKASI
FIREFLY III PERSONAL FINANCE MANAGER

Disusun untuk Memenuhi Tugas Mata Kuliah Arsitektur Perangkat Lunak

Dosen Pengampu:

Edy Suharto S.T., M.Kom



Disusun oleh Kelompok 5:

Ahmad Rafi Alhammam	24060123140138
Elvina Neila Samas	24060123120031
Febrianti Pujiatiningsih	24060123120034
Sophie Venecia May M.	24060123120043

DEPARTEMEN INFORMATIKA
FAKULTAS SAINS DAN MATEMATIKA
UNIVERSITAS DIPONEGORO

2026

BAB I

Deskripsi Program

1.1. Gambaran Umum Firefly III

Firefly III merupakan aplikasi personal finance manager yang bersifat open source dan self-hosted. Aplikasi ini dikembangkan oleh James Cole dan pertama kali dirilis pada tahun 2014. Firefly III dirancang untuk membantu pengguna dalam mengelola keuangan pribadi secara mandiri, mulai dari pencatatan transaksi, pengaturan anggaran, hingga analisis kondisi keuangan secara menyeluruh.

Sebagai aplikasi self-hosted, Firefly III memberikan kontrol penuh kepada pengguna terhadap penyimpanan dan keamanan data keuangan pribadi. Seluruh data dapat disimpan pada server milik pengguna sendiri sehingga risiko kebocoran data ke layanan pihak ketiga dapat diminimalkan. Hal ini menjadikan Firefly III sebagai salah satu solusi manajemen keuangan yang banyak digunakan oleh pengguna yang mengutamakan privasi dan fleksibilitas sistem.

Firefly III mendukung berbagai kebutuhan pengelolaan keuangan seperti pencatatan pemasukan dan pengeluaran, pengelolaan tabungan, pemantauan investasi, serta pembuatan laporan keuangan secara visual. Selain itu, aplikasi ini juga menyediakan fitur otomatisasi transaksi dan integrasi API sehingga dapat dikembangkan lebih lanjut sesuai kebutuhan pengguna. Hingga saat ini, Firefly III menjadi salah satu aplikasi manajemen keuangan populer di GitHub dengan lebih dari 17.000 stars.

1.2. Fitur Utama

Firefly III menyediakan berbagai fitur yang mendukung pengelolaan keuangan secara terstruktur dan efisien. Beberapa fitur utama yang tersedia pada aplikasi ini antara lain sebagai berikut:

a. **Sistem Pembukuan Berpasangan (Double-Entry Bookkeeping)**

Firefly III menggunakan sistem pencatatan transaksi berpasangan sehingga setiap transaksi tercatat secara lebih akurat dan terorganisasi.

b. **Manajemen Anggaran (Budget Management)**

Pengguna dapat membuat dan mengelola anggaran berdasarkan kategori tertentu untuk membantu mengontrol pengeluaran.

c. Laporan dan Visualisasi Data

Aplikasi menyediakan grafik serta laporan interaktif yang menampilkan informasi pendapatan, pengeluaran, saldo, dan statistik keuangan lainnya.

d. Otomatisasi Transaksi

Firefly III mendukung transaksi berulang (recurring transactions) dan sistem aturan otomatis (rule-based automation) untuk mempermudah pengelolaan transaksi.

e. REST API dan Integrasi Sistem

Aplikasi menyediakan REST JSON API, dukungan webhook, serta autentikasi menggunakan OAuth2 dan Personal Access Token untuk kebutuhan integrasi dengan sistem lain.

f. Dukungan Multi-Mata Uang

Firefly III mendukung penggunaan berbagai mata uang sehingga dapat digunakan oleh pengguna dari berbagai negara.

g. Fleksibilitas Deployment

Aplikasi dapat dijalankan menggunakan Docker, Kubernetes, maupun platform self-hosted lainnya sesuai kebutuhan pengguna.

1.3. Informasi Teknis

Firefly III dibangun menggunakan teknologi modern yang mendukung performa, skalabilitas, dan kemudahan pengembangan sistem. Informasi teknis aplikasi ditunjukkan pada tabel berikut.

Atribut	Spesifikasi
Nama Aplikasi	Firefly III
Jenis Aplikasi	Personal Finance Manager
Bahasa Pemrograman	PHP dan JavaScript
Framework Backend	Laravel 11
Framework Frontend	Vue.js 3 dan Bootstrap

Database	MySQL, PostgreSQL, dan SQLite
Lisensi	GNU AGPL v3
Versi Stabil	Firefly III versi 6.x
Popularitas Repository	>17.000 stars di GitHub
Repository Resmi	https://github.com/firefly-iii/firefly-iii

BAB II

Domain

2.1. Domain Aplikasi

Firefly III beroperasi pada domain Financial Technology (FinTech), khususnya pada bidang Personal Finance Management (PFM). Domain ini berfokus pada pengelolaan keuangan pribadi secara digital untuk membantu pengguna dalam mengatur, memantau, dan menganalisis kondisi keuangan secara lebih efektif.

Firefly III dikembangkan sebagai solusi pengelolaan keuangan mandiri yang memungkinkan pengguna mencatat seluruh aktivitas finansial secara terstruktur. Aplikasi ini mendukung berbagai kebutuhan pengelolaan keuangan, mulai dari pencatatan transaksi harian hingga perencanaan keuangan jangka panjang.

Beberapa permasalahan utama yang menjadi fokus dalam domain aplikasi ini antara lain sebagai berikut:

a. Pelacakan Transaksi Keuangan

Pengguna dapat memantau seluruh pemasukan dan pengeluaran secara akurat sehingga kondisi keuangan dapat diketahui dengan lebih jelas.

b. Penyusunan dan Pengelolaan Anggaran

Firefly III menyediakan fitur budgeting untuk membantu pengguna menyusun anggaran berdasarkan kategori tertentu dan mengontrol pengeluaran agar tidak melebihi batas yang direncanakan.

c. Analisis dan Visualisasi Finansial

Aplikasi menyediakan laporan serta grafik interaktif yang membantu pengguna memahami pola pengeluaran dan kondisi finansial secara keseluruhan.

d. Manajemen Keuangan Jangka Panjang

Firefly III mendukung pengelolaan tabungan, utang, piutang, investasi, dan target keuangan sehingga pengguna dapat merencanakan kondisi finansial di masa mendatang.

Sebagai aplikasi pada domain finansial, Firefly III harus memenuhi kebutuhan terhadap integritas data yang tinggi, keamanan informasi yang ketat, serta kemampuan pengelolaan data secara konsisten dan andal. Selain itu, sistem juga dituntut mampu menyajikan informasi keuangan dalam bentuk laporan yang mudah dipahami oleh pengguna.

2.2. Platform

Firefly III dirancang sebagai aplikasi web berbasis server yang bersifat self-hosted. Konsep ini memungkinkan pengguna untuk memasang dan mengelola aplikasi secara mandiri pada infrastruktur yang dimiliki sehingga memberikan fleksibilitas lebih besar dalam pengelolaan sistem dan data.

Aplikasi dapat diakses melalui peramban (browser) pada perangkat desktop maupun perangkat mobile tanpa memerlukan instalasi aplikasi khusus. Antarmuka yang responsif memungkinkan pengguna mengakses fitur-fitur utama secara nyaman pada berbagai ukuran layar.

Dalam proses deployment, Firefly III mendukung berbagai platform dan teknologi modern, antara lain sebagai berikut:

a. Docker dan Kubernetes

Firefly III menyediakan Docker image resmi yang memudahkan proses instalasi dan konfigurasi aplikasi. Selain itu, aplikasi juga mendukung orkestrasi menggunakan Kubernetes untuk kebutuhan skala sistem yang lebih besar.

b. Multi Sistem Operasi

Aplikasi dapat dijalankan pada berbagai lingkungan server seperti Linux, macOS, dan Windows Server.

c. Platform Self-Hosted

Firefly III mendukung integrasi dengan berbagai platform self-hosted dan alat deployment seperti Cloudron, Yunohost, Lando, AMPPS, dan Softaculous.

d. Arsitektur Berbasis Web

Sistem menggunakan arsitektur aplikasi web modern dengan pemisahan antara komponen frontend, backend, dan database sehingga mendukung pengembangan serta pemeliharaan aplikasi secara lebih terstruktur.

Dukungan terhadap berbagai platform dan teknologi tersebut menjadikan Firefly III fleksibel untuk digunakan baik pada lingkungan pengembangan lokal maupun pada server produksi berskala besar.

BAB III

Temuan dan Bukti Gaya Arsitektur

Berdasarkan hasil eksplorasi terhadap source code pada repository resmi Firefly III, ditemukan bahwa Firefly III menerapkan beberapa gaya arsitektur perangkat lunak secara bersamaan (hybrid architecture). Kombinasi arsitektur ini digunakan untuk mendukung kebutuhan sistem yang kompleks, mulai dari pengelolaan transaksi keuangan, integrasi API, otomatisasi proses, hingga skalabilitas aplikasi.

Analisis dilakukan dengan mengamati struktur direktori, pola komunikasi antar komponen, implementasi kode program, serta konfigurasi framework Laravel yang digunakan pada aplikasi.

3.1. Layered Architecture (N-Tier Architecture)

Firefly III menerapkan Layered Architecture sebagai struktur utama aplikasi. Pada arsitektur ini, sistem dibagi ke dalam beberapa lapisan (layer) dengan tanggung jawab yang berbeda. Setiap lapisan hanya berinteraksi dengan lapisan terkait sehingga prinsip separation of concerns dapat terjaga dengan baik.

Pemisahan layer ini membantu meningkatkan keteraturan kode, mempermudah proses pemeliharaan sistem, serta mendukung pengembangan fitur baru secara lebih terstruktur.

Struktur layer pada Firefly III dapat dilihat pada tabel berikut.

Layer	Direktori Utama	Fungsi
Presentation Layer	app/Http/Controllers/	Menangani HTTP request dan response
Business Logic Layer	app/Services/, app/Repositories/	Menjalankan logika bisnis aplikasi
Data Access Layer	app/Models/, database/migrations/	Mengelola akses dan representasi data
Infrastructure Layer	config/, app/Providers/	Mengatur konfigurasi dan layanan sistem

Penerapan arsitektur berlapis terlihat jelas pada mekanisme kerja controller. File pada direktori `app/Http/Controllers/` hanya bertugas menerima request dari pengguna dan meneruskan proses ke layer bisnis tanpa melakukan manipulasi database secara langsung. Proses bisnis utama dijalankan melalui service dan repository yang berada pada direktori `app/Services/` dan `app/Repositories/`.

Firefly III juga menggunakan Repository Pattern untuk memisahkan query database dari logika bisnis aplikasi. Pendekatan ini membuat kode menjadi lebih modular dan mudah dipelihara. Selain itu, penggunaan Dependency Injection melalui provider seperti `FireflyServiceProvider.php` memungkinkan setiap komponen sistem bergantung pada abstraksi, bukan implementasi langsung.

Pada lapisan data, Firefly III menggunakan Eloquent ORM milik Laravel untuk merepresentasikan entitas seperti transaksi, akun, anggaran, dan pengguna. Dengan penggunaan ORM, komunikasi dengan database menjadi lebih terstruktur dan mendukung berbagai sistem database seperti MySQL, PostgreSQL, dan SQLite.

3.2. Client–Server Architecture dengan REST API

Selain menerapkan arsitektur berlapis, Firefly III juga menggunakan Client–Server Architecture. Pada arsitektur ini, pengguna bertindak sebagai client yang mengakses aplikasi melalui browser, sedangkan server menjalankan backend Laravel yang memproses logika bisnis dan komunikasi database.

Penerapan konsep Client–Server diperkuat dengan penggunaan REST JSON API yang memungkinkan integrasi dengan aplikasi eksternal maupun layanan pihak ketiga. Seluruh endpoint API ditempatkan secara khusus pada direktori `app/Api/V1/` sehingga terpisah dari controller untuk antarmuka web biasa.

Beberapa endpoint REST API yang tersedia pada Firefly III ditunjukkan pada tabel berikut.

Endpoint	Fungsi
GET <code>/api/v1/transactions</code>	Mengambil daftar transaksi
POST <code>/api/v1/transactions</code>	Membuat transaksi baru
GET <code>/api/v1/accounts</code>	Mengambil data akun
GET <code>/api/v1/budgets/{id}</code>	Mengambil detail anggaran

DELETE /api/v1/rules/{id}	Menghapus aturan otomatisasi
---------------------------	------------------------------

Untuk menjaga konsistensi data, Firefly III menggunakan transformer pada direktori `app/Transformers/` yang bertugas mengubah objek model menjadi format JSON standar. Sistem autentikasi API juga mendukung OAuth2 dan Personal Access Token sehingga keamanan akses data tetap terjaga.

Pemisahan antara client dan server ini memungkinkan aplikasi diakses secara fleksibel dari berbagai perangkat tanpa perlu instalasi khusus pada sisi pengguna.

3.3. Event-Driven Architecture

Firefly III juga menerapkan konsep Event-Driven Architecture untuk mengurangi keterikatan antar-komponen sistem (loose coupling). Pada pendekatan ini, suatu proses dapat memicu event tertentu yang kemudian akan ditangani oleh listener terkait secara independen.

Penerapan konsep ini terlihat dari penggunaan direktori `app/Events/` dan `app/Listeners/`. Direktori event berisi berbagai kejadian sistem seperti permintaan reset password atau penyimpanan transaksi baru, sedangkan listener bertugas menangani aksi yang harus dijalankan ketika event tersebut dipicu.

Beberapa contoh event dan listener pada Firefly III ditunjukkan pada tabel berikut.

Event	Fungsi
RequestedNewPassword	Memicu proses reset password
WarnAboutBill	Memberikan peringatan tagihan
StoredAccount	Dipicu saat akun berhasil disimpan

Hubungan antara event dan listener diatur pada file `EventServiceProvider.php` sehingga manajemen event menjadi lebih terstruktur dan terpusat.

Firefly III juga menggunakan queue dan background job pada direktori `app/Jobs/` untuk menjalankan proses berat seperti pengiriman email dan webhook secara asinkron. Pendekatan ini membantu meningkatkan performa aplikasi karena request utama pengguna tidak harus menunggu seluruh proses selesai dijalankan.

3.4. Pipe and Filter Architecture (Rule Engine)

Konsep Pipe and Filter Architecture diterapkan pada sistem rule engine yang digunakan untuk otomatisasi transaksi. Setiap transaksi yang masuk akan melewati serangkaian filter dan proses transformasi secara berurutan sebelum menghasilkan output akhir.

Implementasi sistem ini berada pada direktori `app/TransactionRules/`. Folder `Triggers/` berisi berbagai kondisi filter yang digunakan untuk mengevaluasi transaksi, sedangkan folder `Actions/` berisi aksi yang dijalankan ketika kondisi tertentu terpenuhi. Contoh trigger dan action pada Firefly III dapat dilihat pada tabel berikut.

Komponen	Contoh	Fungsi
Trigger	<code>AmountMore</code>	Memeriksa jumlah transaksi
Trigger	<code>DescriptionContains</code>	Memeriksa isi deskripsi transaksi
Action	<code>SetCategory</code>	Menambahkan kategori transaksi
Action	<code>AddTag</code>	Menambahkan tag pada transaksi

Pola pemrosesan berantai ini membuat sistem otomatisasi menjadi fleksibel, modular, dan mudah dikembangkan sesuai kebutuhan pengguna.

3.5. Space-Based Architecture (Parsial)

Firefly III tidak sepenuhnya menerapkan Space-Based Architecture, namun beberapa karakteristik arsitektur ini digunakan untuk mendukung skalabilitas dan pemrosesan paralel.

Salah satu implementasinya terlihat pada dukungan Redis sebagai cache backend yang dikonfigurasi melalui `config/cache.php`. Redis digunakan sebagai shared in-memory data space untuk mempercepat akses data yang sering digunakan.

Selain itu, Firefly III juga mendukung sistem queue yang memungkinkan beberapa worker berjalan secara paralel untuk memproses tugas di background. Session pengguna juga dapat dibagikan antar-server menggunakan Redis sehingga mendukung implementasi load balancing.

Beberapa elemen Space-Based Architecture yang ditemukan ditunjukkan pada tabel berikut.

Komponen	Implementasi
Shared Data Space	Redis Cache
Parallel Processing	Queue Worker
Distributed Session	Redis Session
Separate Processing Unit	firefly-iii/data-importer

Karakteristik tersebut menunjukkan bahwa Firefly III memiliki dukungan terhadap pemrosesan terdistribusi meskipun tidak sepenuhnya mengadopsi Space-Based Architecture secara murni.

BAB IV

Temuan dan Bukti Gaya Arsitektur

Berdasarkan hasil analisis terhadap struktur dan gaya arsitektur Firefly III, ditemukan bahwa Firefly III telah memiliki fondasi arsitektur yang cukup baik melalui penerapan Layered Architecture, REST API, serta berbagai mekanisme modular lainnya. Namun, seiring meningkatnya kebutuhan skalabilitas, integrasi layanan, dan volume data finansial pengguna, terdapat beberapa peluang pengembangan arsitektur yang dapat diterapkan untuk meningkatkan performa, maintainability, efisiensi infrastruktur, dan fleksibilitas sistem di masa mendatang.

4.1. Usulan Microservices Architecture

Saat ini Firefly III masih menggunakan pendekatan arsitektur monolitik (monolithic architecture). Seluruh fungsi utama seperti transaksi, anggaran, laporan, autentikasi, dan webhook dijalankan di dalam satu aplikasi yang sama. Walaupun struktur internal aplikasi sudah cukup terorganisasi melalui Layered Architecture, pendekatan monolitik tetap memiliki keterbatasan ketika sistem berkembang menjadi lebih besar dan kompleks.

Salah satu pengembangan yang dapat diterapkan adalah transformasi bertahap menuju Microservices Architecture. Pada pendekatan ini, setiap layanan utama dipisahkan menjadi service independen yang memiliki tanggung jawab spesifik.

Penerapan microservices memberikan beberapa keuntungan penting. Komponen dengan beban kerja tinggi, seperti pemrosesan transaksi atau analitik laporan keuangan, dapat discala secara independen tanpa harus meningkatkan kapasitas seluruh aplikasi. Selain itu, jika terjadi kegagalan pada satu layanan, seperti webhook atau email, fungsi inti aplikasi tidak akan ikut terganggu sehingga stabilitas sistem tetap terjaga.

Pendekatan ini juga membuka peluang penggunaan teknologi yang lebih sesuai untuk kebutuhan tertentu. Sebagai contoh, analisis data finansial dapat menggunakan Python karena memiliki ekosistem data science yang kuat, sedangkan layanan utama tetap menggunakan PHP Laravel. Pemisahan service juga mendukung pengembangan paralel oleh beberapa tim secara independen.

Rancangan microservices yang diusulkan ditunjukkan pada tabel berikut.

Microservice	Tanggung Jawab	Rekomendasi Teknologi
--------------	----------------	-----------------------

Transaction Service	Mengelola transaksi dan pembukuan	PHP Laravel
Analytics Service	Mengolah laporan dan analisis finansial	Python (FastAPI + Pandas)
Notification Service	Mengirim email dan webhook	Node.js (Express)
Auth Service	Mengelola autentikasi dan OAuth2	Laravel Passport
Import Service	Mengimpor data CSV, OFX, dan bank	Python
API Gateway	Routing request dan rate limiting	Kong / Nginx

Melalui pemisahan service tersebut, sistem diharapkan menjadi lebih fleksibel, scalable, dan mudah dikembangkan pada masa mendatang.

4.2. Usulan CQRS (Command Query Responsibility Segregation)

Firefly III saat ini masih menggunakan model data yang sama untuk menangani proses penulisan data (command) dan pembacaan data (query). Pendekatan ini cukup efektif pada skala kecil hingga menengah, namun dapat menimbulkan bottleneck ketika jumlah transaksi dan proses analitik semakin besar.

Untuk mengatasi hal tersebut, Firefly III dapat mengadopsi pola Command Query Responsibility Segregation (CQRS). Pada pola ini, proses penulisan data dan pembacaan data dipisahkan ke dalam mekanisme yang berbeda.

Penerapan CQRS memungkinkan optimasi performa secara lebih spesifik. Operasi pembacaan data yang kompleks, seperti pembuatan laporan tahunan atau analisis statistik transaksi, dapat menggunakan database atau storage khusus yang dioptimalkan untuk query tanpa membebani database utama. Dengan demikian, proses pencatatan transaksi harian tetap berjalan stabil meskipun terdapat banyak permintaan laporan secara bersamaan.

Selain itu, CQRS juga membantu mengurangi konflik resource antara proses baca dan tulis data. Sistem dapat menggunakan read replica, Redis cache, atau Elasticsearch untuk mempercepat proses query laporan dan visualisasi grafik.

Rancangan implementasi CQRS pada Firefly III dapat dilihat pada tabel berikut.

Komponen	Fungsi
Command Side	Menangani proses penulisan transaksi
Main Database	Menyimpan data utama transaksi
Domain Event	Dipicu setelah proses write berhasil
Event Handler	Memperbarui read model secara asinkron
Read Model	Menyediakan data laporan yang telah dioptimalkan

Dengan penerapan CQRS, performa sistem dapat meningkat terutama pada fitur analitik dan pelaporan finansial dengan volume data yang besar.

4.3. Usulan Serverless / Function as a Service (FaaS)

Beberapa fitur pada Firefly III memiliki karakteristik proses yang berjalan berdasarkan event tertentu dan tidak aktif secara terus-menerus. Kondisi ini membuat pendekatan Serverless atau Function as a Service (FaaS) menjadi salah satu solusi yang potensial.

Pendekatan serverless memungkinkan fungsi tertentu dijalankan hanya ketika dibutuhkan sehingga penggunaan resource menjadi lebih efisien. Selain itu, sistem dapat melakukan auto-scaling secara otomatis berdasarkan jumlah request yang diterima.

Beberapa fitur yang cocok diterapkan menggunakan pendekatan serverless antara lain:

Fitur	Alasan Penggunaan Serverless
Webhook Processing	Bersifat stateless dan dipicu oleh event
Sinkronisasi Kurs Mata Uang	Berjalan terjadwal secara periodik
PDF Report Generation	Membutuhkan resource besar namun jarang digunakan
Email Notification	Diproses secara asynchronous

Implementasi serverless dapat menggunakan platform seperti AWS Lambda atau Cloudflare Workers. Dengan pendekatan ini, biaya operasional infrastruktur dapat ditekan

karena server tidak perlu aktif secara penuh untuk menangani proses yang jarang dijalankan.

Selain itu, isolasi proses pada serverless juga membantu mengurangi dampak kegagalan sistem karena setiap fungsi berjalan secara independen.